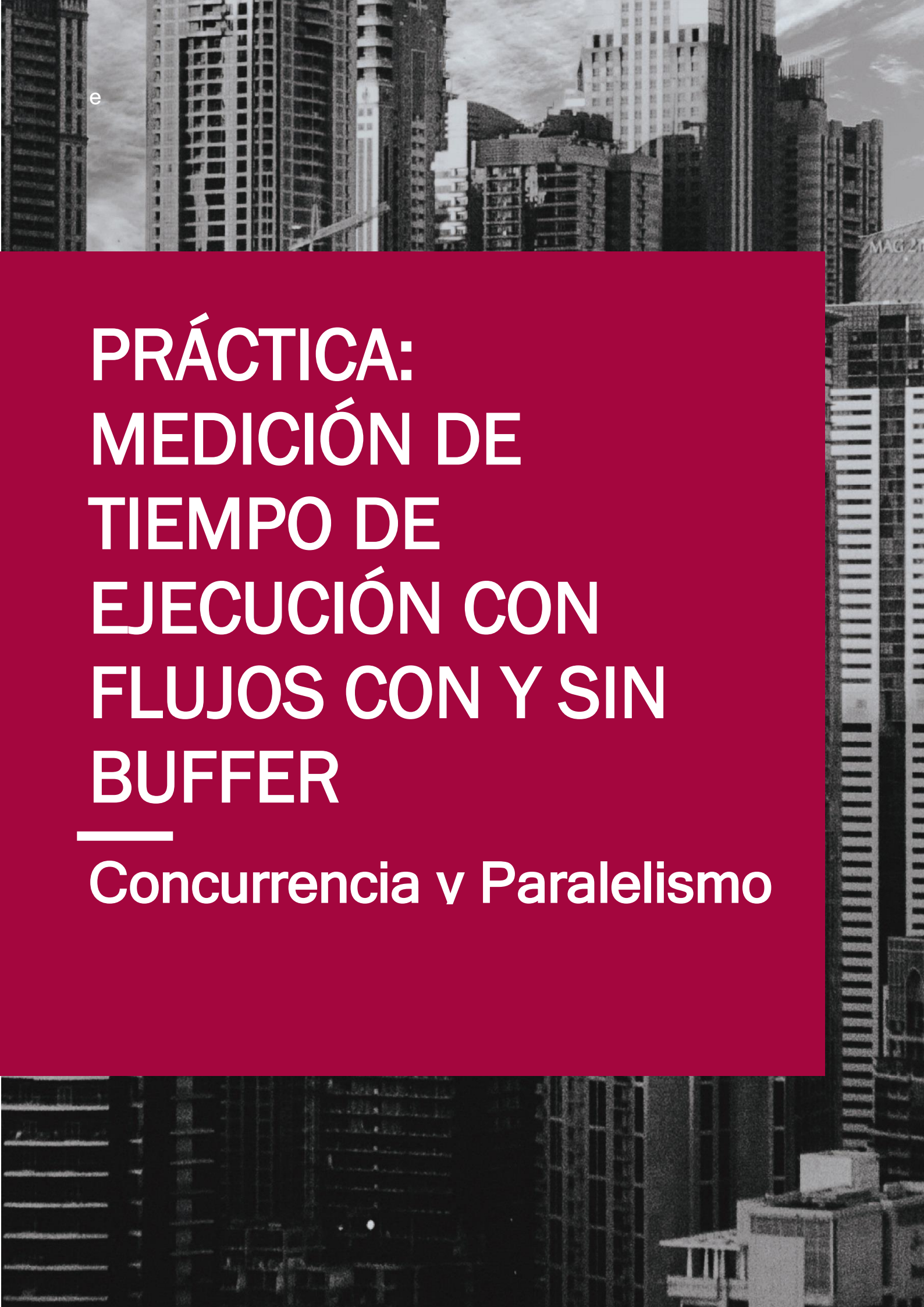


PRÁCTICA: MEDICIÓN DE TIEMPO DE EJECUCIÓN CON FLUJOS CON Y SIN BUFFER

Concurrencia y Paralelismo

Objetivo

Generar/leer archivos de texto y binarios en Java utilizando las clases FileWriter, FileReader, BufferedWriter, BufferedReader, FileOutputStream, BufferedOutputStream, FileInputStream y BufferedInputStream, y medir el tiempo de ejecución del proceso con System.nanoTime().

Marco teórico breve

Flujos de caracteres (texto)
FileWriter: escribe caracteres directamente al disco, operación costosa si se hace carácter por carácter.

BufferedWriter: almacena los caracteres en un buffer interno y los escribe al disco en bloques, reduciendo accesos al sistema de archivos.

FileReader: lee caracteres directamente desde el disco.

BufferedReader: lee caracteres en bloques desde un buffer interno, reduciendo accesos al disco. Además provee readLine().

Flujos de bytes (binario)
FileOutputStream: escribe bytes directamente al disco.

BufferedOutputStream: almacena bytes en un buffer interno (por defecto 8 KB) y los escribe en bloques.

FileInputStream: lee bytes directamente desde el disco.

BufferedInputStream: lee bytes en bloques desde un buffer interno.
Medición

System.nanoTime(): retorna el tiempo actual en nanosegundos con alta precisión; se usa para medir intervalos.

Flujo utilizado	Tamaño (elementos)	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	100,000	16	15	15	15.3333
BufferedWriter	100,000	4	5	5	4.6666
FileReader	100,000	18	18	18	18
BufferedReader	100,000	4	4	4	4
FileOutputStream	1,000,000	2102	2143	2134	2126.3333
BufferedOutputStream	1,000,000	21	20	21	20.6666
FileInputStream	1,000,000	1548	1535	1542	1541.6666
BufferedInputStream	1,000,000	19	19	19	19

```
5   public class EscrituraBuffered {
7       public static void main(String[] args) {
10          String rutaBinario = "archivo_salida.dat";
11
12          System.out.println("Iniciando pruebas con n = " + n + " elementos\n");
13
14          escribirFileWriter(rutaTexto, n);
15          escribirBufferedWriter(rutaTexto, n);
16          leerFileReader(rutaTexto);
```

Problemas 5 | Salida | Consola de depuración | Terminal | Puertos | Run: EscrituraBuffered

```
PS C:\Users\Games> & 'C:\Users\Games\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Games\AppData\Local\Temp\vscodesws_2b22d\jdt_ws\jdt.ls-java-project\bin' 'EscrituraBuffered'
Iniciando pruebas con n = 1000000 elementos

FileWriter:             54 ms
BufferedWriter:         24 ms
FileReader:             41 ms
BufferedReader:         20 ms
-------------------------------------------------
FileOutputStream:       2102 ms
BufferedOutputStream:   21 ms
FileInputStream:        1548 ms
BufferedInputStream:    19 ms
PS C:\Users\Games> & 'C:\Users\Games\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Games\AppData\Local\Temp\vscodesws_2b22d\jdt_ws\jdt.ls-java-project\bin' 'EscrituraBuffered'
Iniciando pruebas con n = 1000000 elementos

FileWriter:             49 ms
BufferedWriter:         21 ms
FileReader:             45 ms
BufferedReader:         21 ms
-------------------------------------------------
FileOutputStream:       2143 ms
BufferedOutputStream:   20 ms
FileInputStream:        1535 ms
BufferedInputStream:    19 ms
PS C:\Users\Games> ^C
PS C:\Users\Games> & 'C:\Users\Games\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Games\AppData\Local\Temp\vscodesws_2b22d\jdt_ws\jdt.ls-java-project\bin' 'EscrituraBuffered'
Iniciando pruebas con n = 1000000 elementos

FileWriter:             49 ms
BufferedWriter:         21 ms
FileReader:             45 ms
BufferedReader:         21 ms
-------------------------------------------------
FileOutputStream:       2134 ms
BufferedOutputStream:   21 ms
FileInputStream:        1542 ms
BufferedInputStream:    19 ms
```

```java
//Erbet Gomez Bohorquez 69338

import java.io.*;

public class EscrituraBuffered {

    Run | Debug
    public static void main(String[] args) {
        int n = 100_000;
        String rutaTexto   = "archivo_salida.txt";
```

```
Problemas 5  Salida  Consola de depuración  Terminal  Puertos                Run: EscrituraBuffered

PS C:\Users\Games> & 'C:\Users\Games\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Games\AppData\Local\Temp\vscodesws_2b22d\jdt_ws\jdt.ls-java-project\bin' 'EscrituraBuffered'
Iniciando pruebas con n = 100000 elementos

FileWriter:             16 ms
BufferedWriter:         4 ms
FileReader:             18 ms
BufferedReader:         4 ms
-------------------------------------------------
FileOutputStream:       229 ms
BufferedOutputStream:   3 ms
FileInputStream:        175 ms
BufferedInputStream:    2 ms
PS C:\Users\Games> & 'C:\Users\Games\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Games\AppData\Local\Temp\vscodesws_2b22d\jdt_ws\jdt.ls-java-project\bin' 'EscrituraBuffered'
Iniciando pruebas con n = 100000 elementos

FileWriter:             15 ms
BufferedWriter:         5 ms
FileReader:             18 ms
BufferedReader:         4 ms
-------------------------------------------------
FileOutputStream:       222 ms
BufferedOutputStream:   3 ms
FileInputStream:        175 ms
BufferedInputStream:    3 ms
PS C:\Users\Games> & 'C:\Users\Games\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Games\AppData\Local\Temp\vscodesws_2b22d\jdt_ws\jdt.ls-java-project\bin' 'EscrituraBuffered'
Iniciando pruebas con n = 100000 elementos

FileWriter:             15 ms
BufferedWriter:         5 ms
FileReader:             18 ms
BufferedReader:         4 ms
-------------------------------------------------
FileOutputStream:       224 ms
BufferedOutputStream:   3 ms
FileInputStream:        177 ms
BufferedInputStream:    2 ms
```

Análisis y preguntas

1. ¿Qué diferencia observaste entre usar flujos con buffer (BufferedWriter, BufferedReader, BufferedOutputStream, BufferedInputStream) y sus equivalentes directos (FileWriter, FileReader, FileOutputStream, FileInputStream)? ¿Por qué ocurre?

Los flujos con buffer son mucho más rápidos que los directos. Por ejemplo, en los flujos binarios (1,000,000 de elementos), el tiempo de escritura bajó de un promedio de 2126.33 ms (con FileOutputStream) a solo 20.66 ms (con BufferedOutputStream). En lectura, bajó de 1541.66 ms a 19 ms. En flujos de texto (100,000 elementos), los tiempos pasaron de 15-18 ms a solo 4 ms al usar buffers.

¿Por qué ocurre?: Las clases sin buffer realizan una llamada de Entrada/Salida (E/S) al disco por cada byte o carácter individual, lo cual es muy lento debido a la latencia del hardware y del sistema operativo. Las clases con buffer almacenan los datos en la memoria RAM y solo interactúan con el disco físico cuando el buffer se llena (trabajando por bloques grandes), reduciendo bastante las operaciones de E/S.

2. ¿Cómo varía el tiempo de ejecución al aumentar n? ¿La relación es lineal? ¿Se comporta igual en lectura que en escritura?

El tiempo aumenta de forma lineal conforme aumenta n. Sin embargo, al usar flujos directos, el costo de cada operación de disco hace que la línea de tiempo se dispare mucho más rápido. Al usar buffers, el crecimiento es lineal pero el impacto de aumentar n es mínimo (escala eficientemente).
¿Se comporta igual en lectura que en escritura? No se comportan exactamente igual; ya que la escritura suele ser ligeramente más lenta que la lectura. En los resultados binarios sin buffer, escribir un millón de elementos tomó 2126 ms, mientras que leerlos tomó 1541 ms. Con buffer, escribir tomó 20.66 ms y leer 19 ms. Esto pasa ya que escribir en disco requiere asignar nuevos bloques de memoria física y actualizar metadatos del archivo.

3. ¿Existe diferencia de rendimiento entre trabajar con caracteres (texto) y con bytes (binario)? ¿A qué se debe?

Si hay bastante diferencia ya que: (100,000 de texto tomaron 4.6 ms en escritura con buffer; 1,000,000 en binario tomaron 20.6 ms), procesar 1 millón de caracteres de texto tardaría unos 46 ms, lo cual es más del doble de lento que el flujo binario (20.6 ms).

¿A qué se debe?: Trabajar con bytes crudos (InputStream/OutputStream) es más rápido porque es una copia directa de datos (ceros y unos). Trabajar con caracteres (Reader/Writer) implica un trabajo de CPU adicional: ya que el programa debe traducir o codificar/decodificar los caracteres del formato interno de Java (UTF-16) al formato de texto del sistema operativo (por ejemplo, UTF-8), añadiendo una pequeña carga de procesamiento.

4. ¿Qué tamaño tiene el buffer por defecto en BufferedOutputStream, BufferedInputStream, BufferedWriter y BufferedReader? ¿Puedes cambiarlo?

El tamaño por defecto del buffer en la mayoría de las implementaciones estándar de Java es de 8,192 bytes (8 KB) para flujos binarios o 8,192 caracteres para texto.
Sí se puede cambiar. Puedes especificar el tamaño del buffer pasando un segundo parámetro (un número entero) al constructor. Por ejemplo: new BufferedReader(new FileReader(ruta), 16384) para un buffer de 16 KB.

5. ¿Por qué BufferedReader ofrece readLine() y BufferedInputStream no? ¿Qué implicación tiene esto al leer texto?

BufferedReader hereda de la familia Reader, que está diseñada específicamente para entender y procesar texto (caracteres). Por lo tanto, es capaz de identificar marcadores semánticos de texto, como los saltos de línea (\n o \r), lo que permite agrupar los caracteres en un String usando readLine().

BufferedInputStream, por otro lado, maneja bytes crudos. No tiene contexto sobre lo que significan esos bytes (podrían ser texto, una imagen, música o un PDF). No puede buscar "saltos de línea" porque el byte equivalente podría ser parte de un píxel, no texto. Implicación: Intentar leer archivos de texto usando flujos de bytes puros (InputStream) es más complejo y propenso a errores (especialmente con caracteres especiales que usan múltiples bytes en UTF-8, los cuales se podrían "cortar" por la mitad). Por eso, siempre se deben usar Reader/Writer para texto plano.

Conclusión

El uso de buffers en Java es lo recomendado especialmente cuando se trabaja con operaciones de I/O en volúmenes medios o grandes de datos. Los resultados demuestran que dejar la lectura/escritura byte por byte directamente al disco compromete severamente el rendimiento de la aplicación debido a los tiempos de latencia del hardware y del sistema operativo. Encapsular los flujos dentro de BufferedReader, BufferedWriter, BufferedInputStream o BufferedOutputStream permite gestionar las transferencias masivas en la memoria RAM de manera transparente y eficiente, obteniendo reducciones de tiempo de ejecución de hasta un 99% sin alterar la lógica fundamental del programa.